So!

In the last few classes we defined the basics of computational complexity theory.

We saw what Turing machines are, how one formalizes the notion of computation.

We took as a definition of "efficient" algorithm, to be one that runs in polynomial time.

We soon came across computational problems that seemed interesting, and which we would like to solve, but for which we could not find any efficient algorithm.

The problems we looked at so far share a certain feature. We can phrase the problem using an existential quantifier.

So, for example, there exists a k-clique in a certain graph; there exists a proof of a certain statement, etc.

It turned that some problems of this form are quite special, in that they can "encode" any other problem of that form.

This is the theory of NP-completeness.

We say that even a problem as simple as 3SAT is NP-complete. So one can express the existence of solution to all kinds of problems (sudoku puzzles, protein folding, what have you) as a problem in satisfiability of propositional formulas.

This is extraordinary! But it is just the beginning of a broad theory of classification. You will get a glimpse of this theory, but it is much vaster than what I can do here.

Today we will talk about space as a computational resource. This is just one of other resources possible.

Broadly speaking, in computational complexity we are interested in the question: "How much of a given computational **resource** is necessary to solve a certain computational **problem** *in a certain **model** of computation*?"

These three concepts are the central unifying theme of the field. Today we will look at:

- How much *memory* (RAM/hard-drive-space/sheets of paper) is necessary to solve a given problem?

But in computational complexity we also look at:

- In a computational model with several processors, how much *inter-processor communication* is necessary?
- Or if certain logical operations cost more *energy* to execute than others (e.g. erasing a bit is actually more energy intensive than flipping two bits x1x2 -> x2x1!), then how much energy does a computation require?
- If two people know parts of the input (Alice knows x, Bob knows y), how much information about their inputs must they reveal to each other in order to implement some joint functionality f(x,y)...
- etc

So the area is vast, but we have to start somewhere.

Today: Space/memory

Definition. Let $(C_1, C_2, \ldots)$ be a computation of a Turing machine. Then the *space* used by the computation is the total number of tape cells which were scanned by any tape head, at any point in time. (Formally it is the size of the set H containing all positions of all tape heads appearing in C_1, C_2, *etc*)

Definition. Let $s : \mathbb{N} \to \mathbb{N}$ be a function. We let

$$DSPACE(s)$$

be the class of all computational problems which can be solved by deterministic Turing machines, such that the computation of machine, on an input of length n, uses at most $s(n)$ space.

Likewise we define $NSPACE(s)$ using non-deterministic Turing machines. Here a subtlety is that the non-deterministic Turing machine should *never* use more than $s(n)$ space, no matter which non-deterministic choices it makes (so this should hold for all computations, not just accepting computations).

Definition. The complexity class "polynomial space", denoted PSPACE, equals $\bigcup_c DSPACE(n^c)$, and likewise we define "non-deterministic polynomial space", NPSPACE, as $\bigcup_c NSPACE(n^c)$.

Theorem. NP $\subseteq$ PSPACE, i.e. every problem solvable in non-deterministic polynomial time can also be solved using only polynomial space!

I would like you to think about this statement for a few minutes and try to prove it in your head.

ideas?

Exactly! We run an algorithm which is broadly called "exhaustive search".

Proof. Let $A \in NP$ be any set in NP. Then let M be a verifier for A, meaning, M is a polynomial-time Turing machine such that $x \in A \iff \exists y \in \{0,1\}^{|x|^c} M(x,y) = 1$. Then consider the following algorithm:

```
input x
for every y of length |x|^c: // how many times, at most, does this loop cycle?
  // at most as many times as the number of different y, which is... 2^(|x|^c)
   run M(x,y); if it halts and outputs 1, then return 1 // this takes time t(n), say
end for
return 0
```

By the way, how much time does this algorithm take? (suppose M runs in time t(n)) Roughly $2^{n^c} t(n)$

Yes? The number of binary strings of length ℓ is..... 2^ℓ? hence the number of times the loop cycles is at most 2^{length-of-y}, yes?

Claim: this algorithm only uses a polynomial amount of memory! Why? because after running M(x,y), I can "reuse" the space used by y, to go to the next y (e.g. by treating y as an n^c-bit number and incrementing it by 1).

Claim 2: the algorithm solves A. *Why?*

Hence $A \in$ PSPACE

Should be obvious: DTIME$(s) \subseteq$ DSPACE$(O(s)) \subseteq$ NSPACE$(O(s)$

why?

first inclusion: meaning, if an algorithm runs in time s(n), then ... how much space does it use? at most O(s(n))! because there is only enough time to move each tape head s times. (more precisely, if M has k tapes, then in time s it can only scan k s(n) cells). So the very same algorithm shows that the problem is in space s.

second inclusion: why? because a non-deterministic Turing machine can run every deterministic algorithm (it's the other way around that's not so easy)

Theorem. $\mathsf{DSPACE}(s(n)) \subseteq \mathsf{DTIME}(2^{O(s)})$, meaning, every problem solvable in space $s(n)$ can be solved in exponential-in-s time.

Having seen the above example, one could conjecture two things: either one can use s space to run for more than 2^s time, or not. One may try to do it, but will fail, because of the following observation:

Crucial observation about space. A Turing machine $M = \langle Q, \Gamma, \delta \rangle$ that uses space s, and has k tapes, can only be in ...

$$how - many?$$

many different configurations.

Let's count:

- The machine can be in $|Q|$ different states, correct?
- Each tape head is in one out of at most s positions, yes? so thats s^k total different ways one can have the tape heads.
- Each cell contains one out of $|\Gamma|$ different symbols. There are at most ks cells being used (s in total, but I don't want to care about which tapes use more and which use less space).

To specify a configuration, I need only to tell you the state, where the tape heads are, and which symbols are there. Hence there are at most

$$|Q|s^k|\Gamma|^{ks}$$

many configurations that use space s!

Everyone understood this?

we're not trying to compute the *exact* number of configurations, we are only giving an upper-bound. Clear?

Ok, so that's an upper bound on the maximum number of configurations that a Turing machine that uses space s can be in.

What happens if, at some point in its computation, the Turing machine enters a configuration C which it has already been in before?

you know, the computation is going:

C1, C2, ... C_t ..... C_{t+u} ....

and $C_t = C_{t+u}$.

Then what happens? It loops! We will see the same subsequence again and again forever!

So if a Turing machine halts, then it has not entered a loop. Ergo, if a Turing machine that uses space s halts, it must do so before having exhausted all possible configurations, i.e., it must halt within time

$$|Q|s^k|\Gamma|^{ks} = 2^{O(s)}$$

e.g. If A is in PSPACE, then it is solved by a Turing machine using space n^c, and this Turing machine always halts, hence it must then halt within time $2^{O(n^c)}$. So PSPACE $\subseteq$ EXP.

That yes, but never more than that.

A small detour

We have defined NP as the class of all problems A such that $x \in A$ can be witnessed by an easy-to-recognize object, meaning

$$x \in A \iff \exists y M(x,y) = 1$$

where y is small and M is efficient.

One needs not stop there. For example, one could consider the class coNP of problems B such that the fact $x \notin B$ can be witnessed by a small, easy-to-recognize object. I.e. a problem B is in coNP iff its complement $A = \{0,1\}^* \setminus B$ is in NP. Or, in other words, there exists a verifier M such that

$$x \in B \iff \forall y \in \{0,1\}^{n^c} M(x,y) = 1$$

($= 1, = 0$ doesnt change the definition because M is deterministic, so it can always negate the output)

For example, the set PRIMES of prime numbers is in coNP.

$$x \text{ is prime} \iff \forall 1 < a,b < x \qquad a \cdot b \neq x.$$

But we do not need to sop here, either. We could consider predicates that have more sophisticated quantification.

For example, I give you as input some Turing machine F, which computes some function, a length n, a time-bound t, and a value K, and I want to know if the minimum value that F takes on inputs of length n is at least K:

$$MIN = \{\langle F, 0^t, 0^n, K\rangle \mid F \text{ is TM, and } \min_{x \in \{0,1\}^n} M(x) \geq K, \text{ or } F \text{ runs for longer than } t \text{ on such an input}\}$$

$$\langle F, 0^t, 0^n, K\rangle \in MIN \iff \exists x \forall y \quad K \leq M(x) \leq M(y))$$

So knowing if the minimum of a function is large is naturally a $\exists\forall$ predicate.

Claim. $MIN \in$ PSPACE... *why?*

The following algorithm computes MIN and runs in polynomial space:

```
input F, 0^t, 0^n, K
/// what to do?
for every possible x of length n
   for every possible y of length n
      run M on x and run M on y for at most t steps, and compare
         if F(x) > f(y), then x is not a minimum, so move onto the next x
         if f(x) < K, then the minimum (whatever it is) is less than K, so return NO //
here
      if test succeeded for current x and all y, return YES
if reached end and didn't find a good x, return NO
```

In other words, the algorithm just runs over all x, and

No, it works I think. If we suppose that the minimum is the first x (meaning $x = 0^n$), it will return NO "here" if f(x) < K, and will return YES otherwise

OK, this class ($\exists\forall$) is called Σ_2. We could define a similar class $\exists\forall\exists\forall$, for example, which would be Σ_4.

But also $\forall\exists$ is Π_2 and $\forall\exists\forall$ is Π_3, etcetera.

In this notation, NP $= \Sigma_1$ and coNP $= \Pi_1$.

and ALL OF THESE CLASSES ARE IN PSPACE!!

Meaning, in PSPACE I can decide any predicate with a bounded number of quantifiers, over an efficiently computable predicate. (Homework)

But I can do even more.

Definition. The set TQBF, of *True, Quantified Boolean formulas*, is the set of all CNFs $\psi(x_1, y_1, \ldots, x_n, y_n)$ on $2n$ variables such that the statement:

$$\exists x_1 \forall y_1 \ldots \exists x_n \forall y_n \psi(x_1, y_1, \ldots, x_n, y_n)$$

is a true statement.

Claim. TQBF is in PSPACE

why? by doing 2n nested for cycles. (homework)

Definition. A set A is $PSPACE$-hard if for every $B \in$ PSPACE there exists a polytime computable function $f : \{0,1\}^* \to \{0,1\}^*$ such that

$$x \in B \iff f(x) \in A$$

i.e. if $B \leq_p A$ for all B in PSPACE.

A set is PSPACE complete iff it is in PSPACE and it is PSPACE-hard.

Theorem. TQBF is PSPACE-complete.

(Because every set (other than $\{0,1\}^*$ and $\varnothing$) is PSPACE-hard if f is allowed to run in PSPACE (the reduction can just solve B and output an element in or out of the set)

Proof of this (in 10-15m sorry)

Definition. Let M be a non-deterministic turing machine, and s a number (the space used). Let us define a **directed** graph G, as follows. The nodes of G are the configurations of M that use at most s space.

A configuration C in G is has an edge to another configuration C' of G if and only if one of the transition functions of M transforms C into C' in a single step of computation.

Given an input x, the non-deterministic machine M accepts input x using space $\leq s$ if and only if there exists a path, in G, between the initial configuration on input x, and some configuration which accepts. correct?

Of course, this path can be very long (exp(s)-long), so what can we do?

We can apply recursion. Consider the following recursive algorithm:

```
exists-path(s,t,m) // should output 1 if there exists a path in G
                   //from s to t of length at most 2^m
    for every w node of G // think of 2 as a node in the middle
       if exists-path(s,w,m-1) and exists-path(w,t,m-1), output TRUE
    end for
    // if we haven't found a path
    output FALSE
```

This algorithm decides if there exists a path between s and t, but notice how it works.

exists-path(s,t,w) iff exists middle node w such that **for both before/after** exists a middle node w'...

The issue here is whether one can unroll this definition systematically, so that after all the unfolding we get a simple, polynomial-time predicate.
And the answer, not obvious, is that YES, we can.

details in book :-)

ok now for the kicker.

Consider a two-player game. Imagine it is played on an nxn board, and we limit the number of moves to be polynomial.

Which player wins? Well, the first player wins if there exists a move that he can make, such that no matter how the adversary responds, there exists a respose that he can choose, so no matter how the adversary responds, etc

So the existence of an optimal strategy for the first player is a predicate of the form

$$\exists move \forall response \exists move \forall response \ldots . Player 1 wins$$

so it turns out that this is not a coincidence, and one will find that finding optimal strategies for most two-player games is a PSPACE-complete problem! (e.g. nxn checkers, nxn chess, and many many more)

Under the belief that PSPACE $\not\subseteq$ NP, we are then led to the conclusion that Chess (a PSPACE-complete problem) is harder than Sudoku (an NP-complete problem)!

and although perhaps this is intuitive, there is no other way, that I know, of meaningfully making such a statement in a precise way

Ok that's the class :-)

No wait, that statement I can't make :-) Who knows?

What we are saying is that, if we generalize the two games, chess and sudoku, to any size ($n \times n$ chess, $n \times n$ sudoku), then the computational problem we obtain is provably harder (in that it is hard for a larger class) for chess than it is for sudoku

about the finite cases that we actually play, it's hard to say anything in terms of complexity, because all finite problems can be solved in finite time :-)

There are actually people who did their careers thinking about the computational complexity of games

So for example super mario is NP-hard and things like this are actually theorems that you can prove!